

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Кафедра «Компьютерной безопасности»

Отчет к лабораторной работе №9
по дисциплине
«Языки программирования»

Работу выполнил студент группы СКБ253

подпись, дата

О.Д. Родин

Работу проверил

подпись, дата

С.А. Булгаков

Содержание

Постановка задачи	3
Алгоритм решения задачи	4
1. Описание структуры программы	4
1.1 Общая информация	4
1.2 Типы данных и приватные поля	4
1.3 Конструкторы и деструктор	4
1.4 Доступ к элементам.....	5
1.5 Итераторы	5
1.6 Управление памятью.....	5
1.7 Модификаторы	6
2. Проверка базовой функциональности	6
Приложение А.....	8
Файл vector.hpp	8
Приложение Б	16
Файл test.cpp	16
Диаграмма классов	32

Постановка задачи

Разработать шаблонный класс `kb::Vector<T>` — полнофункциональный аналог стандартного контейнера `std::vector` из стандартной библиотеки C++. Реализация должна быть выполнена на языке C++ в соответствии со стандартом C++98 (`-std=c++98`).

Класс должен поддерживать следующий функционал:

- конструкторы: по умолчанию, с указанием размера и значения, копирующий, из диапазона итераторов;
- деструктор с корректным освобождением памяти;
- оператор присваивания и перегрузки метода `assign` (по значению и по диапазону итераторов);
- доступ к элементам: `at()`, `operator[]`, `front()`, `back()`, `data()`;
- итераторы прямого хода: `begin()`, `end()` в константном и неконстантном вариантах;
- обратные итераторы: `rbegin()`, `rend()` через `std::reverse_iterator`;
- методы управления памятью: `empty()`, `size()`, `max_size()`, `reserve()`, `capacity()`, `shrink_to_fit()`;
- модификаторы: `clear()`, `insert()` (одиночный элемент и диапазон), `erase()` (одиночный и диапазон), `push_back()`, `pop_back()`, `resize()`, `swap()`;
- свободная функция `swap()` как `friend`.

Для верификации реализации необходимо разработать набор автоматических тестов с использованием фреймворка Google Test (GTest), охватывающих все публичные методы, граничные случаи и смешанные сценарии. Сборка проекта выполняется через CMake с автоматической загрузкой GTest v1.14.0 через FetchContent.

Алгоритм решения задачи

1. Описание структуры программы

1.1 Общая информация

Программа представляет собой шаблонный класс `kb::Vector<T>`, реализованный целиком в заголовочном файле `vector.hpp` в пространстве имён `kb`. Класс управляет динамически выделяемым массивом объектов типа `T` с автоматическим удвоением ёмкости при нехватке места. Внутреннее состояние хранится в пяти полях: три итератора (`__data_`, `__begin_`, `__end_`) и два счётчика (`__size_`, `__capacity_`).

1.2 Типы данных и приватные поля

Класс определяет восемь публичных псевдонимов типов в соответствии со стандартом контейнеров STL: `iterator`, `const_iterator`, `value_type`, `size_type`, `reference`, `const_reference`, `reverse_iterator` (через `std::reverse_iterator<iterator>`), `const_reverse_iterator` (через `std::reverse_iterator<const_iterator>`). Реализация приведена в листинге 1.

Листинг 1 — Типы данных и поля класса

```
namespace kb {
template<typename T>
class Vector {
public:
    typedef T* iterator;
    typedef const T* const_iterator;
    typedef T value_type;
    typedef size_t size_type;
    typedef value_type& reference;
    typedef const value_type& const_reference;
    typedef std::reverse_iterator<iterator> reverse_iterator;
    typedef std::reverse_iterator<const_iterator> const_reverse_iterator;
private:
    iterator __begin_;
    iterator __end_;
    iterator __data_;
    size_type __size_;
    size_type __capacity_;
};
```

1.3 Конструкторы и деструктор

Конструктор по умолчанию инициализирует все указатели нулём, `size` и `capacity` равны 0. Конструктор `Vector(size_type, value_type)` выделяет массив заданного размера с помощью `placement new` и инициализирует каждый элемент. Копирующий конструктор создаёт глубокую копию. Конструктор из диапазона итераторов `Vector(const_iterator, const_iterator)` вычисляет размер как `last-first` и копирует элементы. Деструктор освобождает память через `delete[]`. Реализации показаны в листинге 2.

Листинг 2 — Конструктор из диапазона итераторов

```
template<typename T>
Vector<T>::Vector(const_iterator __first, const_iterator __last) {
    __size_ = __last - __first;
    __capacity_ = __size_;
    iterator buffer = new value_type[__capacity_ * sizeof(value_type)];
    __data_ = new (buffer) value_type();
    for (size_type i = 0; i < __size_; ++i) {
        new (&(__data_ + i)) value_type(*(__first + i));
    }
    __begin_ = __data_;
    __end_ = __data_ + __size_;
}
```

1.4 Доступ к элементам

Метод `at()` выполняет доступ с проверкой границ и выбрасывает `std::runtime_error` при выходе за пределы. Оператор `operator[]` также проверяет границы. Методы `front()` и `back()` возвращают ссылки на первый и последний элементы через `__begin_` и `__end_-1`. Метод `data()` возвращает указатель на внутренний буфер. Все методы доступны в константном и неконстантном вариантах. Реализация `at()` приведена в листинге 3.

Листинг 3 — Метод `at()`

```
template<typename T>
typename Vector<T>::reference Vector<T>::at(size_type __pos) {
    if (__pos < 0 || __pos >= __size_)
        throw std::runtime_error("Could not find element - invalid index");
    return *(__data_ + __pos);
}
```

1.5 Итераторы

Класс предоставляет четыре пары итераторов. Прямые итераторы `begin()/end()` возвращают `__begin_` и `__end_` соответственно — в константном и неконстантном вариантах. Обратные итераторы `rbegin()/rend()` реализованы через `std::reverse_iterator<iterator>`, оборачивая `__end_` и `__begin_`. Это обеспечивает совместимость с алгоритмами STL. Реализация приведена в листинге 4.

Листинг 4 — Обратные итераторы

```
template<typename T>
typename Vector<T>::reverse_iterator Vector<T>::rbegin() {
    return reverse_iterator(__end_);
}
template<typename T>
typename Vector<T>::reverse_iterator Vector<T>::rend() {
    return reverse_iterator(__begin_);
}
```

1.6 Управление памятью

Метод `reserve(size_type)` перевыделяет память, если запрошенная ёмкость превышает текущую: сохраняет старый указатель, выделяет новый буфер, копирует элементы placement new, освобождает старый буфер и обновляет `__begin_`, `__end_`, `__capacity_`. Метод `shrink_to_fit()` уменьшает `capacity` до `size` аналогичным образом. Метод `max_size()` вычисляется как `static_cast<size_type>(-1) / sizeof(T)`. Реализация `reserve()` приведена в листинге 5.

Листинг 5 — Метод `reserve()`

```
template<typename T>
void Vector<T>::reserve(size_type __new_capacity) {
    if (__new_capacity > max_size()) throw std::length_error("Vector");
    if (__new_capacity <= __capacity_) return;
    iterator old_data = __data_;
    iterator buffer = new value_type[__new_capacity * sizeof(value_type)];
    __data_ = new (buffer) value_type();
    __begin_ = __end_ = nullptr;
    __capacity_ = __new_capacity;
    for (size_type i = 0; i < __size_; ++i)
        new (&(*(__data_ + i))) value_type(*(old_data + i));
    delete[] old_data;
    __begin_ = __data_;
    __end_ = __data_ + __size_;
}
```

1.7 Модификаторы

Метод `push_back()` добавляет элемент в конец; при переполнении удваивает ёмкость через `reserve()`. Метод `pop_back()` уменьшает `size` на 1, смещая `__end_`. Метод `resize()` при увеличении вызывает `reserve()` и инициализирует новые элементы значением по умолчанию, при уменьшении только корректирует `__size_` и `__end_`. Методы `insert(iterator, value)` и `insert(iterator, first, last)` смещают существующие элементы вправо и вставляют новые. Методы `erase(iterator)` и `erase(first, last)` смещают элементы влево. Метод `swap()` обменивает все пять внутренних полей двух объектов. Реализация `insert()` приведена в листинге 6.

Листинг 6 — Метод `insert()` (одиночный элемент)

```
template<typename T>
typename Vector<T>::iterator
Vector<T>::insert(iterator __pos, const_reference __value) {
    size_type index = __pos - __begin_;
    if (__size_ == __capacity_) {
        size_type new_cap = (__capacity_ == 0) ? 1 : __capacity_ * 2;
        reserve(new_cap);
    }
    iterator pos = __begin_ + index;
    for (iterator it = __end_; it != pos; --it)
        *it = *(it - 1);
    *pos = __value;
    ++__size_;
    __end_ = __data_ + __size_;
    return pos;
}
```

2. Проверка базовой функциональности

Для верификации реализации написан файл `test.cpp` с тестами Google Test, охватывающими следующие группы:

- Constructors — конструктор по умолчанию, с размером и значением, копирующий, независимость копии;
- Destructor — отсутствие утечек памяти;
- AssignmentOperator — базовое присваивание, самоприсваивание, независимость;
- Assign — метод `assign()` по значению и по диапазону итераторов;
- ElementAccess — `at()`, `operator[]`, `front()`, `back()`, `data()` в константном и неконстантном вариантах;
- Iterators, ConstIterators — прямые итераторы, арифметика указателей;
- ReverseIterators, ConstReverseIterators — обратный обход, изменение через `rbegin`;
- IteratorRangeConstructor — конструирование из диапазона;
- IteratorRangeAssign — `assign` из диапазона итераторов;
- Capacity — `empty`, `size`, `max_size`, `reserve`, `shrink_to_fit`;
- Modifiers — `clear`, `push_back`, `pop_back`, `resize`, `swap`;
- InsertSingle, InsertRange — вставка в начало, конец, середину, перераспределение памяти;
- EraseSingle, EraseRange — удаление первого, последнего, среднего элемента, диапазонов;
- NonMemberSwap — свободная функция `swap()`;
- Mixed, MixedNew — комбинированные сценарии с `insert/erase/reverse-iterate`.

Тест `Constructors::Default` (листинг 7) проверяет начальное состояние вектора.

Листинг 7 — Пример теста: `Constructors::Default`

```
TEST(Constructors, Default) {
```

```
kb::Vector<int> v;  
EXPECT_TRUE(v.empty());  
EXPECT_EQ(v.size(), 0u);  
EXPECT_EQ(v.capacity(), 0u);  
}
```

Тест `InsertSingle::InMiddle` (листинг 8) проверяет корректность сдвига элементов.

Листинг 8 — Пример теста: `InsertSingle::InMiddle`

```
TEST(InsertSingle, InMiddle) {  
    kb::Vector<int> v;  
    v.push_back(1); v.push_back(3);  
    v.insert(v.begin() + 1, 2);  
    ASSERT_EQ(v.size(), 3u);  
    EXPECT_EQ(v[0], 1);  
    EXPECT_EQ(v[1], 2);  
    EXPECT_EQ(v[2], 3);  
}
```

Приложение А

(Код основной программы)

Файл vector.hpp

```
#ifndef VECTOR_HPP
#define VECTOR_HPP

#include <cstdint>
#include <cmath>
#include <exception>
#include <iterator>

namespace kb {
template<typename T>
class Vector {
public:
    /* types */
    typedef T* iterator;
    typedef const T* const_iterator;
    typedef T value_type;
    typedef size_t size_type;
    typedef value_type& reference;
    typedef const value_type& const_reference;
    typedef std::reverse_iterator<iterator> reverse_iterator;
    typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

    /* constructors : default, copy, iterators */
    Vector(); // * DONE
    Vector(size_type __size, value_type __value = value_type()); // * DONE
    Vector(const Vector& __other); // * DONE
    Vector(const_iterator __first, const_iterator __last); // * DONE

    /* destructor */
    ~Vector(); // * DONE

    /* operators */
    Vector& operator=(const Vector& __other); // * DONE
    void assign(size_type __size, const_reference __value); // * DONE
    void assign(const_iterator __first, const_iterator __last); // * DONE

    /* element access */
    reference at(size_type __pos); // * DONE
    const_reference at(size_type __pos) const; // * DONE
    reference operator[](size_type __pos); // * DONE
    const_reference operator[](size_type __pos) const; // * DONE
    reference front(); // * DONE
    const_reference front() const; // * DONE
    reference back(); // * DONE
    const_reference back() const; // * DONE
    iterator data(); // * DONE
    iterator data() const; // * DONE

    /* iterators */
    iterator begin(); // * DONE
    const_iterator begin() const; // * DONE
    iterator end(); // * DONE
    const_iterator end() const; // * DONE
    reverse_iterator rbegin(); // * DONE
    const_reverse_iterator rbegin() const; // * DONE
    reverse_iterator rend(); // * DONE
    const_reverse_iterator rend() const; // * DONE

    /* memory */
    bool empty() const; // * DONE
    size_type size() const; // * DONE
    size_type max_size() const; // * DONE
    void reserve(size_type __size); // * DONE
    size_type capacity() const; // * DONE
    void shrink_to_fit(); // * DONE
};
}
```



```

/* methods for working with elements */
void      clear(); // * DONE
iterator  insert(iterator __pos, const_reference __value); // * DONE
void      insert(iterator __pos, const_iterator __first, const_iterator __last); // * DONE
iterator  erase(iterator __pos); // * DONE
iterator  erase(iterator __first, iterator __last); // * DONE
void      push_back(value_type value); // * DONE
void      pop_back(); // * DONE
void      resize(size_type __size); // * DONE
void      swap(Vector& __other); // * DONE

private:
    iterator  __begin_;
    iterator  __end_;
    iterator  __data_;
    size_type __size_;
    size_type __capacity_;
};

template<typename T>
Vector<T>::Vector() : __begin_(nullptr), __end_(nullptr), __data_(nullptr), __size_(0),
__capacity_(0) {}

template<typename T>
Vector<T>::Vector(size_type __size, value_type __value) {
    __size_ = __size;
    __capacity_ = __size_;

    iterator buffer = new value_type[__capacity_ * sizeof(value_type)];
    __data_ = new (buffer) value_type();
    for (size_type i = 0; i < __size_; ++i) {
        new (&(__data_ + i)) value_type(__value);
    }

    __begin_ = __data_;
    __end_ = __data_ + __size_;
}

template<typename T>
Vector<T>::Vector(const Vector& __other) {
    __size_ = __other.__size_;
    __capacity_ = __other.__capacity_;

    iterator buffer = new value_type[__other.__capacity_ * sizeof(value_type)];
    __data_ = new (buffer) value_type();
    for (size_type i = 0; i < __size_; ++i) {
        new (&(__data_ + i)) value_type(*(__other.__data_ + i));
    }

    __begin_ = __data_;
    __end_ = __data_ + __size_;
}

template<typename T>
Vector<T>::Vector(const_iterator __first, const_iterator __last) {
    __size_ = __last - __first;
    __capacity_ = __size_;

    iterator buffer = new value_type[__capacity_ * sizeof(value_type)];
    __data_ = new (buffer) value_type();
    for (size_type i = 0; i < __size_; ++i) {
        new (&(__data_ + i)) value_type(*(__first + i));
    }

    __begin_ = __data_;
    __end_ = __data_ + __size_;
}

template<typename T>
Vector<T>::~~Vector() {
    /*for (size_type i = __size_ - 1; i >= 0; --i) {
        __data_[i]->~T();
    }::operator delete(__data_);*/
    delete[] __data_;
}

```

```

    __data_ = nullptr;
    __begin_ = nullptr;
    __end_ = nullptr;
}

template<typename T>
Vector<T>& Vector<T>::operator=(const Vector& __other) {
    if (this != &__other) {
        delete[] __data_;
        __data_ = nullptr;
        __begin_ = nullptr;
        __end_ = nullptr;

        __size_ = __other.__size_;
        __capacity_ = __other.__capacity_;

        iterator buffer = new value_type[__capacity_ * sizeof(value_type)];
        __data_ = new (buffer) value_type();
        for (size_type i = 0; i < __size_; ++i) {
            new (&(__data_ + i)) value_type(*(__other.__data_ + i));
        }

        __begin_ = __data_;
        __end_ = __data_ + __size_;
    }

    return *this;
}

template<typename T>
void Vector<T>::assign(size_type __size, const_reference __value) {
    delete[] __data_;
    __data_ = nullptr;
    __begin_ = nullptr;
    __end_ = nullptr;

    __size_ = __size;
    __capacity_ = __size;

    iterator buffer = new value_type[__capacity_ * sizeof(value_type)];
    __data_ = new (buffer) value_type();
    for (size_type i = 0; i < __size_; ++i) {
        new (&(__data_ + i)) value_type(__value);
    }

    __begin_ = __data_;
    __end_ = __data_ + __size_;
}

template<typename T>
void Vector<T>::assign(const_iterator __first, const_iterator __last) {
    delete[] __data_;
    __data_ = nullptr;
    __begin_ = nullptr;
    __end_ = nullptr;

    __size_ = __last - __first;
    __capacity_ = __size_;

    iterator buffer = new value_type[__capacity_ * sizeof(value_type)];
    __data_ = new (buffer) value_type();
    for (size_type i = 0; i < __size_; ++i) {
        new (&(__data_ + i)) value_type(*(__first + i));
    }

    __begin_ = __data_;
    __end_ = __data_ + __size_;
}

template<typename T>
typename Vector<T>::reference Vector<T>::at(size_type __pos) {
    if (__pos < 0 || __pos >= __size_)
        throw std::runtime_error("Could not find element - invalid index");
}

```

```

    return *(__data_ + __pos);
}

template<typename T>
typename Vector<T>::const_reference Vector<T>::at(size_type __pos) const {
    if (__pos < 0 || __pos >= __size_)
        throw std::runtime_error("Could not find element - invalid index");
    return *(__data_ + __pos);
}

template<typename T>
typename Vector<T>::reference Vector<T>::operator[](size_type __pos) {
    if (__pos < 0 || __pos >= __size_)
        throw std::runtime_error("Could not find element - invalid index");
    return *(__data_ + __pos);
}

template<typename T>
typename Vector<T>::const_reference Vector<T>::operator[](size_type __pos) const {
    if (__pos < 0 || __pos >= __size_)
        throw std::runtime_error("Could not find element - invalid index");
    return *(__data_ + __pos);
}

template<typename T>
typename Vector<T>::reference Vector<T>::front() {
    return *(__begin_);
}

template<typename T>
typename Vector<T>::const_reference Vector<T>::front() const {
    return *(__begin_);
}

template<typename T>
typename Vector<T>::reference Vector<T>::back() {
    return *(__end_ - 1);
}

template<typename T>
typename Vector<T>::const_reference Vector<T>::back() const {
    return *(__end_ - 1);
}

template<typename T>
typename Vector<T>::iterator Vector<T>::data() {
    return __data_;
}

template<typename T>
typename Vector<T>::iterator Vector<T>::data() const {
    return __data_;
}

template<typename T>
typename Vector<T>::iterator Vector<T>::begin() {
    return __begin_;
}

template<typename T>
typename Vector<T>::const_iterator Vector<T>::begin() const {
    return __begin_;
}

template<typename T>
typename Vector<T>::iterator Vector<T>::end() {
    return __end_;
}

template<typename T>
typename Vector<T>::const_iterator Vector<T>::end() const {
    return __end_;
}

```

```

template<typename T>
typename Vector<T>::reverse_iterator Vector<T>::rbegin() {
    return reverse_iterator(__end_);
}

template<typename T>
typename Vector<T>::const_reverse_iterator Vector<T>::rbegin() const {
    return const_reverse_iterator(__end_);
}

template<typename T>
typename Vector<T>::reverse_iterator Vector<T>::rend() {
    return reverse_iterator(__begin_);
}

template<typename T>
typename Vector<T>::const_reverse_iterator Vector<T>::rend() const {
    return const_reverse_iterator(__begin_);
}

template<typename T>
bool Vector<T>::empty() const {
    return __size_ == 0;
}

template<typename T>
typename Vector<T>::size_type Vector<T>::size() const {
    return __size_;
}

template<typename T>
typename Vector<T>::size_type Vector<T>::max_size() const {
    return static_cast<size_type>(-1) / sizeof(T);
}

template<typename T>
void Vector<T>::reserve(size_type __new_capacity) {
    if (__new_capacity > max_size())
        throw std::length_error("Vector");
    if (__new_capacity <= __capacity_) return;

    iterator old_data = __data_;
    iterator buffer = new value_type[__capacity_ * sizeof(value_type)];
    __data_ = new (buffer) value_type();

    __begin_ = __end_ = nullptr;
    __capacity_ = __new_capacity;

    for (size_type i = 0; i < __size_; ++i) {
        new (&(__data_ + i)) value_type(*(old_data + i));
    }

    delete[] old_data;
    __begin_ = __data_;
    __end_ = __data_ + __size_;
}

template<typename T>
typename Vector<T>::size_type Vector<T>::capacity() const {
    return __capacity_;
}

template<typename T>
void Vector<T>::shrink_to_fit() {
    if (__size_ < __capacity_) {
        iterator old_data = __data_;
        __capacity_ = __size_;

        iterator buffer = new value_type[__capacity_ * sizeof(value_type)];
        __data_ = new (buffer) value_type();
    }
}

```

```

    __begin_ = __end_ = nullptr;

    for (size_type i = 0; i < __size_; ++i) {
        new (&(__data_ + i)) value_type(*(old_data + i));
    }

    delete[] old_data;
    __begin_ = __data_;
    __end_ = __data_ + __size_;
}

template<typename T>
void Vector<T>::clear() {
    if (__data_ != nullptr) {
        delete[] __data_;
        __data_ = nullptr;
        __begin_ = nullptr;
        __end_ = nullptr;
    }
    __size_ = 0;
}

template<typename T>
typename Vector<T>::iterator Vector<T>::insert(iterator __pos, const_reference __value) {
    size_type index = __pos - __begin_;

    if (__size_ == __capacity_) {
        size_type new_capacity = (__capacity_ == 0) ? 1 : __capacity_ * 2;
        reserve(new_capacity);
    }

    iterator pos = __begin_ + index;

    for (iterator it = __end_; it != pos; --it) {
        *it = *(it - 1);
    }

    *pos = __value;
    ++__size_;
    __end_ = __data_ + __size_;

    return pos;
}

template<typename T>
void Vector<T>::insert(iterator __pos, const_iterator __first, const_iterator __last) {
    size_type index = __pos - __begin_;
    size_type count = __last - __first;
    if (count == 0) return;

    while (__size_ + count > __capacity_) {
        size_type new_capacity = (__capacity_ == 0) ? 1 : __capacity_ * 2;
        reserve(new_capacity);
    }

    iterator pos = __begin_ + index;

    for (iterator it = __end_ + count - 1; it != pos + count - 1; --it) {
        *it = *(it - count);
    }

    for (size_type i = 0; i < count; ++i) {
        *(pos + i) = *(__first + i);
    }

    __size_ += count;
    __end_ = __data_ + __size_;
}

template<typename T>
typename Vector<T>::iterator Vector<T>::erase(iterator __pos) {

```

```

    for (iterator it = __pos; it != __end_ - 1; ++it) {
        *it = *(it + 1);
    }

    --__size_;
    __end_ = __data_ + __size_;

    return __pos;
}

template<typename T>
typename Vector<T>::iterator Vector<T>::erase(iterator __first, iterator __last) {
    size_type count = __last - __first;
    if (count == 0) return __first;

    for (iterator it = __first; it != __end_ - count; ++it) {
        *it = *(it + count);
    }

    __size_ -= count;
    __end_ = __data_ + __size_;

    return __first;
}

template<typename T>
void Vector<T>::push_back(value_type __value) {
    if (__size_ + 1 > max_size())
        throw std::length_error("Vector");

    if (__size_ == __capacity_) {
        size_type new_capacity = (__capacity_ == 0) ? 1 : __capacity_ * 2;
        reserve(new_capacity);
    }

    *(__data_ + (__size_++)) = __value;
    __end_ = __data_ + __size_;
}

template<typename T>
void Vector<T>::pop_back() {
    __end_ -= 1;
    __size_ -= 1;
}

template<typename T>
void Vector<T>::resize(size_type __size) {
    if (__size == __size_) {
        return;
    }

    if (__size > __capacity_) {
        if (__size > max_size())
            throw std::length_error("Vector");

        size_type old_size = __size_;
        size_type new_capacity = std::pow(2, std::ceil(std::log2(__size)));
        reserve(new_capacity);

        // Initialize new elements with default value
        for (size_type i = old_size; i < __size; ++i) {
            *(__data_ + i) = value_type();
        }
    }

    __size_ = __size;
    __end_ = __data_ + __size_;
}

template<typename T>
void Vector<T>::swap(Vector& __other) {
    // ! that looks like trash, but it works...

```

```

if (__other.__begin_ == nullptr && __other.__end_ == nullptr)
    throw std::runtime_error("Vector");

iterator cnt_data      = __data_;
iterator cnt_begin     = __begin_;
iterator cnt_end       = __end_;
size_type cnt_size     = __size_;
size_type cnt_capacity = __capacity_;

__data_      = __other.__data_;
__begin_     = __other.__begin_;
__end_       = __other.__end_;
__size_      = __other.__size_;
__capacity_  = __other.__capacity_;

__other.__data_      = cnt_data;
__other.__begin_     = cnt_begin;
__other.__end_       = cnt_end;
__other.__size_      = cnt_size;
__other.__capacity_  = cnt_capacity;
}

template<typename T>
void swap(Vector<T>& __a, Vector<T>& __b) {
    __a.swap(__b);
}

} // namespace kb

#endif /* VECTOR_HPP */

```

Приложение Б

(Тесты)

Файл test.cpp

```
#include <gtest/gtest.h>
#include "vector.hpp"

// === Constructors ===

TEST(Constructors, Default) {
    kb::Vector<int> v;
    EXPECT_TRUE(v.empty());
    EXPECT_EQ(v.size(), 0u);
    EXPECT_EQ(v.capacity(), 0u);
}

TEST(Constructors, SizeAndValue) {
    kb::Vector<int> v(5, 42);
    ASSERT_EQ(v.size(), 5u);
    EXPECT_EQ(v.capacity(), 5u);
    for (size_t i = 0; i < v.size(); ++i) {
        EXPECT_EQ(v[i], 42);
    }
}

TEST(Constructors, SizeDefaultValue) {
    kb::Vector<int> v(3);
    ASSERT_EQ(v.size(), 3u);
    for (size_t i = 0; i < v.size(); ++i) {
        EXPECT_EQ(v[i], 0);
    }
}

TEST(Constructors, Copy) {
    kb::Vector<int> v(3, 10);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;

    kb::Vector<int> copy(v);
    ASSERT_EQ(copy.size(), v.size());
    for (size_t i = 0; i < v.size(); ++i) {
        EXPECT_EQ(copy[i], v[i]);
    }
}

TEST(Constructors, CopyIsIndependent) {
    kb::Vector<int> v(3, 10);
    kb::Vector<int> copy(v);
    copy[0] = 999;
    EXPECT_EQ(v[0], 10);
    EXPECT_EQ(copy[0], 999);
}

// === Destructor ===

TEST(Destructor, NoLeaks) {
    kb::Vector<int>* v = new kb::Vector<int>(100, 5);
    delete v;
}

// === Assignment Operator ===

TEST(AssignmentOperator, Basic) {
    kb::Vector<int> v(3, 10);
    kb::Vector<int> v2;
    v2 = v;
    ASSERT_EQ(v2.size(), 3u);
    for (size_t i = 0; i < v2.size(); ++i) {
```



```

        EXPECT_EQ(v2[i], 10);
    }
}

TEST(AssignmentOperator, SelfAssignment) {
    kb::Vector<int> v(3, 10);
    kb::Vector<int>& ref = v;
    v = ref;
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(v[0], 10);
}

TEST(AssignmentOperator, Independence) {
    kb::Vector<int> v(3, 10);
    kb::Vector<int> v2;
    v2 = v;
    v2[0] = 999;
    EXPECT_EQ(v[0], 10);
    EXPECT_EQ(v2[0], 999);
}

// === Assign ===

TEST(Assign, Basic) {
    kb::Vector<int> v;
    v.assign(5, 7);
    ASSERT_EQ(v.size(), 5u);
    for (size_t i = 0; i < v.size(); ++i) {
        EXPECT_EQ(v[i], 7);
    }
}

TEST(Assign, Overwrite) {
    kb::Vector<int> v(3, 10);
    v.assign(2, 20);
    ASSERT_EQ(v.size(), 2u);
    EXPECT_EQ(v[0], 20);
    EXPECT_EQ(v[1], 20);
}

// === Element Access ===

TEST(ElementAccess, At) {
    kb::Vector<int> v(3);
    v[0] = 10;
    v[1] = 20;
    v[2] = 30;
    EXPECT_EQ(v.at(0), 10);
    EXPECT_EQ(v.at(1), 20);
    EXPECT_EQ(v.at(2), 30);
}

TEST(ElementAccess, AtThrowsOutOfRange) {
    kb::Vector<int> v(3);
    EXPECT_THROW(v.at(3), std::runtime_error);
    EXPECT_THROW(v.at(100), std::runtime_error);
}

TEST(ElementAccess, AtConst) {
    kb::Vector<int> v(2, 5);
    const kb::Vector<int>& cv = v;
    EXPECT_EQ(cv.at(0), 5);
    EXPECT_EQ(cv.at(1), 5);
    EXPECT_THROW(cv.at(2), std::runtime_error);
}

TEST(ElementAccess, SubscriptOperator) {
    kb::Vector<int> v(3);
    v[0] = 100;
    v[1] = 200;
    v[2] = 300;
    EXPECT_EQ(v[0], 100);
    EXPECT_EQ(v[1], 200);
    EXPECT_EQ(v[2], 300);
}

```

```

TEST(ElementAccess, SubscriptConst) {
    kb::Vector<int> v(2, 42);
    const kb::Vector<int>& cv = v;
    EXPECT_EQ(cv[0], 42);
    EXPECT_EQ(cv[1], 42);
}

TEST(ElementAccess, Front) {
    kb::Vector<int> v(3);
    v[0] = 11;
    v[1] = 22;
    v[2] = 33;
    EXPECT_EQ(v.front(), 11);
}

TEST(ElementAccess, FrontConst) {
    kb::Vector<int> v(2, 99);
    const kb::Vector<int>& cv = v;
    EXPECT_EQ(cv.front(), 99);
}

TEST(ElementAccess, Back) {
    kb::Vector<int> v(3);
    v[0] = 11;
    v[1] = 22;
    v[2] = 33;
    EXPECT_EQ(v.back(), 33);
}

TEST(ElementAccess, BackConst) {
    kb::Vector<int> v(2, 77);
    const kb::Vector<int>& cv = v;
    EXPECT_EQ(cv.back(), 77);
}

TEST(ElementAccess, Data) {
    kb::Vector<int> v(3, 5);
    int* p = v.data();
    EXPECT_EQ(p[0], 5);
    EXPECT_EQ(p[1], 5);
    EXPECT_EQ(p[2], 5);
}

TEST(ElementAccess, DataConst) {
    kb::Vector<int> v(2, 8);
    const kb::Vector<int>& cv = v;
    int* p = cv.data();
    EXPECT_EQ(p[0], 8);
    EXPECT_EQ(p[1], 8);
}

// === Iterators ===

TEST(Iterators, BeginEnd) {
    kb::Vector<int> v(3);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;

    int* it = v.begin();
    EXPECT_EQ(*it, 1);
    ++it;
    EXPECT_EQ(*it, 2);
    ++it;
    EXPECT_EQ(*it, 3);
    ++it;
    EXPECT_EQ(it, v.end());
}

TEST(Iterators, BeginEqualsEndForEmpty) {
    kb::Vector<int> v;
    EXPECT_EQ(v.begin(), v.end());
}

```

```

TEST(Iterators, IterateWithPointerArithmetic) {
    kb::Vector<int> v(4, 0);
    v[0] = 10;
    v[1] = 20;
    v[2] = 30;
    v[3] = 40;

    int sum = 0;
    for (int* it = v.begin(); it != v.end(); ++it) {
        sum += *it;
    }
    EXPECT_EQ(sum, 100);
}

TEST(Iterators, RandomAccess) {
    kb::Vector<int> v(5, 0);
    for (size_t i = 0; i < 5; ++i) {
        v[i] = static_cast<int>(i) * 10;
    }

    int* it = v.begin();
    EXPECT_EQ(*(it + 0), 0);
    EXPECT_EQ(*(it + 1), 10);
    EXPECT_EQ(*(it + 2), 20);
    EXPECT_EQ(*(it + 3), 30);
    EXPECT_EQ(*(it + 4), 40);

    EXPECT_EQ(v.end() - v.begin(), 5);
}

// === Capacity ===

TEST(Capacity, Empty) {
    kb::Vector<int> v;
    EXPECT_TRUE(v.empty());

    kb::Vector<int> v2(3, 1);
    EXPECT_FALSE(v2.empty());
}

TEST(Capacity, Size) {
    kb::Vector<int> v;
    EXPECT_EQ(v.size(), 0u);

    kb::Vector<int> v2(10, 1);
    EXPECT_EQ(v2.size(), 10u);
}

TEST(Capacity, MaxSize) {
    kb::Vector<int> v;
    EXPECT_GT(v.max_size(), 0u);
}

TEST(Capacity, Reserve) {
    kb::Vector<int> v;
    v.reserve(100);
    EXPECT_GE(v.capacity(), 100u);
    EXPECT_EQ(v.size(), 0u);
}

TEST(Capacity, ReservePreservesData) {
    kb::Vector<int> v(3);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;
    v.reserve(100);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
    EXPECT_EQ(v.size(), 3u);
}

```

```

TEST(Capacity, CapacityAfterConstruction) {
    kb::Vector<int> v(5, 1);
    EXPECT_GE(v.capacity(), 5u);
}

TEST(Capacity, ShrinkToFit) {
    kb::Vector<int> v(3, 1);
    v.reserve(100);
    EXPECT_GE(v.capacity(), 100u);
    v.shrink_to_fit();
    EXPECT_EQ(v.capacity(), v.size());
}

TEST(Capacity, ShrinkToFitPreservesData) {
    kb::Vector<int> v(3);
    v[0] = 10;
    v[1] = 20;
    v[2] = 30;
    v.reserve(100);
    v.shrink_to_fit();
    EXPECT_EQ(v[0], 10);
    EXPECT_EQ(v[1], 20);
    EXPECT_EQ(v[2], 30);
}

// === Modifiers ===

TEST(Modifiers, Clear) {
    kb::Vector<int> v(5, 42);
    v.clear();
    EXPECT_TRUE(v.empty());
    EXPECT_EQ(v.size(), 0u);
}

TEST(Modifiers, PushBack) {
    kb::Vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
}

TEST(Modifiers, PushBackGrowsCapacity) {
    kb::Vector<int> v;
    for (int i = 0; i < 20; ++i) {
        v.push_back(i);
    }
    ASSERT_EQ(v.size(), 20u);
    EXPECT_GE(v.capacity(), 20u);
    for (int i = 0; i < 20; ++i) {
        EXPECT_EQ(v[i], i);
    }
}

TEST(Modifiers, PopBack) {
    kb::Vector<int> v(3);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;
    v.pop_back();
    EXPECT_EQ(v.size(), 2u);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
}

TEST(Modifiers, ResizeGrow) {
    kb::Vector<int> v(2, 5);
    v.resize(5);
    ASSERT_EQ(v.size(), 5u);
    EXPECT_EQ(v[0], 5);
    EXPECT_EQ(v[1], 5);
    EXPECT_EQ(v[2], 0);
    EXPECT_EQ(v[3], 0);

```

```

    EXPECT_EQ(v[4], 0);
}

TEST(Modifiers, ResizeShrink) {
    kb::Vector<int> v(5, 10);
    v.resize(2);
    EXPECT_EQ(v.size(), 2u);
    EXPECT_EQ(v[0], 10);
    EXPECT_EQ(v[1], 10);
}

TEST(Modifiers, ResizeSameSize) {
    kb::Vector<int> v(3, 7);
    v.resize(3);
    EXPECT_EQ(v.size(), 3u);
    EXPECT_EQ(v[0], 7);
}

TEST(Modifiers, Swap) {
    kb::Vector<int> a(3, 1);
    kb::Vector<int> b(5, 2);
    a.swap(b);
    ASSERT_EQ(a.size(), 5u);
    ASSERT_EQ(b.size(), 3u);
    for (size_t i = 0; i < a.size(); ++i) {
        EXPECT_EQ(a[i], 2);
    }
    for (size_t i = 0; i < b.size(); ++i) {
        EXPECT_EQ(b[i], 1);
    }
}

// === Mixed scenarios ===

TEST(Mixed, PushBackThenPopBack) {
    kb::Vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    v.pop_back();
    ASSERT_EQ(v.size(), 2u);
    EXPECT_EQ(v.back(), 20);
}

TEST(Mixed, ClearChecksEmpty) {
    kb::Vector<int> v(5, 42);
    EXPECT_FALSE(v.empty());
    v.clear();
    EXPECT_TRUE(v.empty());
    EXPECT_EQ(v.size(), 0u);
}

TEST(Mixed, AssignThenResize) {
    kb::Vector<int> v;
    v.assign(3, 100);
    v.resize(5);
    ASSERT_EQ(v.size(), 5u);
    EXPECT_EQ(v[0], 100);
    EXPECT_EQ(v[1], 100);
    EXPECT_EQ(v[2], 100);
}

TEST(Mixed, DoubleType) {
    kb::Vector<double> v;
    v.push_back(1.5);
    v.push_back(2.5);
    v.push_back(3.5);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_DOUBLE_EQ(v[0], 1.5);
    EXPECT_DOUBLE_EQ(v[1], 2.5);
    EXPECT_DOUBLE_EQ(v[2], 3.5);
}

// === Iterator Range Constructor ===

```

```

TEST(IteratorRangeConstructor, FromArray) {
    int arr[] = {10, 20, 30, 40, 50};
    kb::Vector<int> v(arr, arr + 5);
    ASSERT_EQ(v.size(), 5u);
    EXPECT_EQ(v[0], 10);
    EXPECT_EQ(v[1], 20);
    EXPECT_EQ(v[2], 30);
    EXPECT_EQ(v[3], 40);
    EXPECT_EQ(v[4], 50);
}

TEST(IteratorRangeConstructor, FromVectorIterators) {
    kb::Vector<int> src(3, 0);
    src[0] = 1;
    src[1] = 2;
    src[2] = 3;

    kb::Vector<int> v(src.begin(), src.end());
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
}

TEST(IteratorRangeConstructor, EmptyRange) {
    int arr[] = {1};
    kb::Vector<int> v(arr, arr);
    EXPECT_TRUE(v.empty());
    EXPECT_EQ(v.size(), 0u);
}

TEST(IteratorRangeConstructor, SingleElement) {
    int arr[] = {42};
    kb::Vector<int> v(arr, arr + 1);
    ASSERT_EQ(v.size(), 1u);
    EXPECT_EQ(v[0], 42);
}

TEST(IteratorRangeConstructor, SubRange) {
    int arr[] = {10, 20, 30, 40, 50};
    kb::Vector<int> v(arr + 1, arr + 4);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(v[0], 20);
    EXPECT_EQ(v[1], 30);
    EXPECT_EQ(v[2], 40);
}

TEST(IteratorRangeConstructor, IsIndependentFromSource) {
    int arr[] = {1, 2, 3};
    kb::Vector<int> v(arr, arr + 3);
    arr[0] = 999;
    EXPECT_EQ(v[0], 1);
}

// === Iterator Range Assign ===

TEST(IteratorRangeAssign, FromArray) {
    int arr[] = {5, 10, 15};
    kb::Vector<int> v;
    v.assign(arr, arr + 3);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(v[0], 5);
    EXPECT_EQ(v[1], 10);
    EXPECT_EQ(v[2], 15);
}

TEST(IteratorRangeAssign, OverwriteExisting) {
    kb::Vector<int> v(5, 99);
    int arr[] = {1, 2};
    v.assign(arr, arr + 2);
    ASSERT_EQ(v.size(), 2u);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
}

```

```

TEST(IteratorRangeAssign, EmptyRange) {
    kb::Vector<int> v(3, 10);
    int arr[] = {1};
    v.assign(arr, arr);
    EXPECT_EQ(v.size(), 0u);
    EXPECT_TRUE(v.empty());
}

TEST(IteratorRangeAssign, FromVectorIterators) {
    kb::Vector<int> src(4, 0);
    src[0] = 10;
    src[1] = 20;
    src[2] = 30;
    src[3] = 40;

    kb::Vector<int> v;
    v.assign(src.begin(), src.end());
    ASSERT_EQ(v.size(), 4u);
    for (size_t i = 0; i < v.size(); ++i) {
        EXPECT_EQ(v[i], src[i]);
    }
}

TEST(IteratorRangeAssign, SingleElement) {
    int arr[] = {77};
    kb::Vector<int> v(10, 0);
    v.assign(arr, arr + 1);
    ASSERT_EQ(v.size(), 1u);
    EXPECT_EQ(v[0], 77);
}

// === Const Iterators ===

TEST(ConstIterators, BeginEndConst) {
    kb::Vector<int> v(3, 0);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;
    const kb::Vector<int>& cv = v;

    const int* it = cv.begin();
    EXPECT_EQ(*it, 1);
    ++it;
    EXPECT_EQ(*it, 2);
    ++it;
    EXPECT_EQ(*it, 3);
    ++it;
    EXPECT_EQ(it, cv.end());
}

TEST(ConstIterators, EmptyConstVector) {
    const kb::Vector<int> cv;
    EXPECT_EQ(cv.begin(), cv.end());
}

TEST(ConstIterators, IterateConstVector) {
    kb::Vector<int> v(4, 0);
    v[0] = 10;
    v[1] = 20;
    v[2] = 30;
    v[3] = 40;
    const kb::Vector<int>& cv = v;

    int sum = 0;
    for (const int* it = cv.begin(); it != cv.end(); ++it) {
        sum += *it;
    }
    EXPECT_EQ(sum, 100);
}

TEST(ConstIterators, ConstIteratorDistance) {
    kb::Vector<int> v(5, 1);
    const kb::Vector<int>& cv = v;
    EXPECT_EQ(cv.end() - cv.begin(), 5);
}

```

```

// === Reverse Iterators ===

TEST(ReverseIterators, RbeginRend) {
    kb::Vector<int> v(3, 0);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;

    kb::Vector<int>::reverse_iterator rit = v.rbegin();
    EXPECT_EQ(*rit, 3);
    ++rit;
    EXPECT_EQ(*rit, 2);
    ++rit;
    EXPECT_EQ(*rit, 1);
    ++rit;
    EXPECT_TRUE(rit == v.rend());
}

TEST(ReverseIterators, EmptyVector) {
    kb::Vector<int> v;
    EXPECT_TRUE(v.rbegin() == v.rend());
}

TEST(ReverseIterators, SingleElement) {
    kb::Vector<int> v(1, 42);
    kb::Vector<int>::reverse_iterator rit = v.rbegin();
    EXPECT_EQ(*rit, 42);
    ++rit;
    EXPECT_TRUE(rit == v.rend());
}

TEST(ReverseIterators, ModifyThroughReverseIterator) {
    kb::Vector<int> v(3, 0);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;

    *v.rbegin() = 99;
    EXPECT_EQ(v[2], 99);
}

TEST(ReverseIterators, ReverseSum) {
    kb::Vector<int> v(4, 0);
    v[0] = 10;
    v[1] = 20;
    v[2] = 30;
    v[3] = 40;

    int sum = 0;
    for (kb::Vector<int>::reverse_iterator rit = v.rbegin(); rit != v.rend(); ++rit) {
        sum += *rit;
    }
    EXPECT_EQ(sum, 100);
}

// === Const Reverse Iterators ===

TEST(ConstReverseIterators, RbeginRendConst) {
    kb::Vector<int> v(3, 0);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;
    const kb::Vector<int>& cv = v;

    kb::Vector<int>::const_reverse_iterator rit = cv.rbegin();
    EXPECT_EQ(*rit, 3);
    ++rit;
    EXPECT_EQ(*rit, 2);
    ++rit;
    EXPECT_EQ(*rit, 1);
    ++rit;
    EXPECT_TRUE(rit == cv.rend());
}

```



```

TEST(ConstReverseIterators, EmptyConstVector) {
    const kb::Vector<int> cv;
    EXPECT_TRUE(cv.rbegin() == cv.rend());
}

TEST(ConstReverseIterators, ConstReverseSum) {
    kb::Vector<int> v(4, 0);
    v[0] = 10;
    v[1] = 20;
    v[2] = 30;
    v[3] = 40;
    const kb::Vector<int>& cv = v;

    int sum = 0;
    for (kb::Vector<int>::const_reverse_iterator rit = cv.rbegin(); rit != cv.rend(); ++rit) {
        sum += *rit;
    }
    EXPECT_EQ(sum, 100);
}

// === Insert (single element) ===

TEST(InsertSingle, AtBeginning) {
    kb::Vector<int> v(3, 0);
    v[0] = 2;
    v[1] = 3;
    v[2] = 4;

    kb::Vector<int>::iterator it = v.insert(v.begin(), 1);
    ASSERT_EQ(v.size(), 4u);
    EXPECT_EQ(*it, 1);
    EXPECT_EQ(it, v.begin());
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
    EXPECT_EQ(v[3], 4);
}

TEST(InsertSingle, AtEnd) {
    kb::Vector<int> v(3, 0);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;

    kb::Vector<int>::iterator it = v.insert(v.end(), 4);
    ASSERT_EQ(v.size(), 4u);
    EXPECT_EQ(*it, 4);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
    EXPECT_EQ(v[3], 4);
}

TEST(InsertSingle, InMiddle) {
    kb::Vector<int> v(3, 0);
    v[0] = 1;
    v[1] = 3;
    v[2] = 4;

    kb::Vector<int>::iterator it = v.insert(v.begin() + 1, 2);
    ASSERT_EQ(v.size(), 4u);
    EXPECT_EQ(*it, 2);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
    EXPECT_EQ(v[3], 4);
}

TEST(InsertSingle, IntoEmptyVector) {
    kb::Vector<int> v;
    kb::Vector<int>::iterator it = v.insert(v.begin(), 42);
    ASSERT_EQ(v.size(), 1u);
    EXPECT_EQ(*it, 42);
}

```

```

    EXPECT_EQ(v[0], 42);
}

TEST(InsertSingle, TriggersReallocation) {
    kb::Vector<int> v;
    v.push_back(1);
    v.push_back(3);
    size_t old_cap = v.capacity();

    while (v.size() < v.capacity()) {
        v.push_back(99);
    }
    // vector is now full
    v.insert(v.begin() + 1, 2);
    EXPECT_GT(v.capacity(), old_cap);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
}

TEST(InsertSingle, MultipleInserts) {
    kb::Vector<int> v;
    v.insert(v.begin(), 3);
    v.insert(v.begin(), 2);
    v.insert(v.begin(), 1);
    v.insert(v.end(), 4);
    v.insert(v.end(), 5);
    ASSERT_EQ(v.size(), 5u);
    for (size_t i = 0; i < 5; ++i) {
        EXPECT_EQ(v[i], static_cast<int>(i) + 1);
    }
}

// === Insert (range) ===

TEST(InsertRange, AtBeginning) {
    kb::Vector<int> v(3, 0);
    v[0] = 3;
    v[1] = 4;
    v[2] = 5;

    int arr[] = {1, 2};
    v.insert(v.begin(), arr, arr + 2);
    ASSERT_EQ(v.size(), 5u);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
    EXPECT_EQ(v[3], 4);
    EXPECT_EQ(v[4], 5);
}

TEST(InsertRange, AtEnd) {
    kb::Vector<int> v(3, 0);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;

    int arr[] = {4, 5};
    v.insert(v.end(), arr, arr + 2);
    ASSERT_EQ(v.size(), 5u);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
    EXPECT_EQ(v[3], 4);
    EXPECT_EQ(v[4], 5);
}

TEST(InsertRange, InMiddle) {
    kb::Vector<int> v(2, 0);
    v[0] = 1;
    v[1] = 4;

    int arr[] = {2, 3};
    v.insert(v.begin() + 1, arr, arr + 2);
    ASSERT_EQ(v.size(), 4u);
    EXPECT_EQ(v[0], 1);

```

```

    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
    EXPECT_EQ(v[3], 4);
}

TEST(InsertRange, EmptyRange) {
    kb::Vector<int> v(3, 10);
    int arr[] = {1};
    v.insert(v.begin(), arr, arr);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(v[0], 10);
}

TEST(InsertRange, IntoEmptyVector) {
    kb::Vector<int> v;
    int arr[] = {1, 2, 3};
    v.insert(v.begin(), arr, arr + 3);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
}

TEST(InsertRange, SingleElementRange) {
    kb::Vector<int> v(2, 0);
    v[0] = 1;
    v[1] = 3;

    int arr[] = {2};
    v.insert(v.begin() + 1, arr, arr + 1);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
}

TEST(InsertRange, TriggersReallocation) {
    kb::Vector<int> v;
    v.push_back(1);
    // fill to capacity
    while (v.size() < v.capacity()) {
        v.push_back(0);
    }
    size_t old_cap = v.capacity();

    int arr[] = {10, 20, 30, 40, 50};
    v.insert(v.end(), arr, arr + 5);
    EXPECT_GT(v.capacity(), old_cap);
    EXPECT_EQ(v[0], 1);
}

TEST(InsertRange, LargeRangeInsert) {
    kb::Vector<int> v;
    v.push_back(0);
    v.push_back(99);

    int arr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    v.insert(v.begin() + 1, arr, arr + 10);
    ASSERT_EQ(v.size(), 12u);
    EXPECT_EQ(v[0], 0);
    for (int i = 1; i <= 10; ++i) {
        EXPECT_EQ(v[i], i);
    }
    EXPECT_EQ(v[11], 99);
}

// === Erase (single element) ===

TEST(EraseSingle, First) {
    kb::Vector<int> v(3, 0);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;

```

```

        kb::Vector<int>::iterator it = v.erase(v.begin());
        ASSERT_EQ(v.size(), 2u);
        EXPECT_EQ(*it, 2);
        EXPECT_EQ(it, v.begin());
        EXPECT_EQ(v[0], 2);
        EXPECT_EQ(v[1], 3);
    }

    TEST(EraseSingle, Last) {
        kb::Vector<int> v(3, 0);
        v[0] = 1;
        v[1] = 2;
        v[2] = 3;

        kb::Vector<int>::iterator it = v.erase(v.end() - 1);
        ASSERT_EQ(v.size(), 2u);
        EXPECT_EQ(it, v.end());
        EXPECT_EQ(v[0], 1);
        EXPECT_EQ(v[1], 2);
    }

    TEST(EraseSingle, Middle) {
        kb::Vector<int> v(4, 0);
        v[0] = 1;
        v[1] = 2;
        v[2] = 3;
        v[3] = 4;

        kb::Vector<int>::iterator it = v.erase(v.begin() + 1);
        ASSERT_EQ(v.size(), 3u);
        EXPECT_EQ(*it, 3);
        EXPECT_EQ(v[0], 1);
        EXPECT_EQ(v[1], 3);
        EXPECT_EQ(v[2], 4);
    }

    TEST(EraseSingle, OnlyElement) {
        kb::Vector<int> v(1, 42);
        kb::Vector<int>::iterator it = v.erase(v.begin());
        EXPECT_EQ(v.size(), 0u);
        EXPECT_TRUE(v.empty());
        EXPECT_EQ(it, v.end());
    }

    TEST(EraseSingle, SequentialEraseFromFront) {
        kb::Vector<int> v(5, 0);
        for (size_t i = 0; i < 5; ++i) {
            v[i] = static_cast<int>(i) + 1;
        }

        v.erase(v.begin());
        v.erase(v.begin());
        ASSERT_EQ(v.size(), 3u);
        EXPECT_EQ(v[0], 3);
        EXPECT_EQ(v[1], 4);
        EXPECT_EQ(v[2], 5);
    }

    TEST(EraseSingle, SequentialEraseFromBack) {
        kb::Vector<int> v(5, 0);
        for (size_t i = 0; i < 5; ++i) {
            v[i] = static_cast<int>(i) + 1;
        }

        v.erase(v.end() - 1);
        v.erase(v.end() - 1);
        ASSERT_EQ(v.size(), 3u);
        EXPECT_EQ(v[0], 1);
        EXPECT_EQ(v[1], 2);
        EXPECT_EQ(v[2], 3);
    }

    // === Erase (range) ===

```

```

TEST(EraseRange, EntireVector) {
    kb::Vector<int> v(5, 42);
    kb::Vector<int>::iterator it = v.erase(v.begin(), v.end());
    EXPECT_EQ(v.size(), 0u);
    EXPECT_TRUE(v.empty());
    EXPECT_EQ(it, v.end());
}

TEST(EraseRange, FromBeginning) {
    kb::Vector<int> v(5, 0);
    for (size_t i = 0; i < 5; ++i) {
        v[i] = static_cast<int>(i) + 1;
    }

    kb::Vector<int>::iterator it = v.erase(v.begin(), v.begin() + 2);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(*it, 3);
    EXPECT_EQ(it, v.begin());
    EXPECT_EQ(v[0], 3);
    EXPECT_EQ(v[1], 4);
    EXPECT_EQ(v[2], 5);
}

TEST(EraseRange, FromEnd) {
    kb::Vector<int> v(5, 0);
    for (size_t i = 0; i < 5; ++i) {
        v[i] = static_cast<int>(i) + 1;
    }

    kb::Vector<int>::iterator it = v.erase(v.begin() + 3, v.end());
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(it, v.end());
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
}

TEST(EraseRange, FromMiddle) {
    kb::Vector<int> v(5, 0);
    for (size_t i = 0; i < 5; ++i) {
        v[i] = static_cast<int>(i) + 1;
    }

    kb::Vector<int>::iterator it = v.erase(v.begin() + 1, v.begin() + 4);
    ASSERT_EQ(v.size(), 2u);
    EXPECT_EQ(*it, 5);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 5);
}

TEST(EraseRange, EmptyRange) {
    kb::Vector<int> v(3, 10);
    kb::Vector<int>::iterator it = v.erase(v.begin() + 1, v.begin() + 1);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(*it, 10);
    EXPECT_EQ(v[0], 10);
    EXPECT_EQ(v[1], 10);
    EXPECT_EQ(v[2], 10);
}

TEST(EraseRange, SingleElementRange) {
    kb::Vector<int> v(3, 0);
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;

    kb::Vector<int>::iterator it = v.erase(v.begin() + 1, v.begin() + 2);
    ASSERT_EQ(v.size(), 2u);
    EXPECT_EQ(*it, 3);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 3);
}

// === Non-member swap ===

```

```

TEST(NonMemberSwap, Basic) {
    kb::Vector<int> a(3, 1);
    kb::Vector<int> b(5, 2);
    kb::swap(a, b);
    ASSERT_EQ(a.size(), 5u);
    ASSERT_EQ(b.size(), 3u);
    for (size_t i = 0; i < a.size(); ++i) {
        EXPECT_EQ(a[i], 2);
    }
    for (size_t i = 0; i < b.size(); ++i) {
        EXPECT_EQ(b[i], 1);
    }
}

TEST(NonMemberSwap, DifferentCapacities) {
    kb::Vector<int> a(2, 10);
    kb::Vector<int> b(8, 20);
    size_t cap_a = a.capacity();
    size_t cap_b = b.capacity();
    kb::swap(a, b);
    EXPECT_EQ(a.capacity(), cap_b);
    EXPECT_EQ(b.capacity(), cap_a);
    EXPECT_EQ(a.size(), 8u);
    EXPECT_EQ(b.size(), 2u);
}

// === Mixed scenarios with new methods ===

TEST(MixedNew, InsertThenErase) {
    kb::Vector<int> v;
    v.push_back(1);
    v.push_back(3);
    v.push_back(5);
    v.insert(v.begin() + 1, 2);
    v.insert(v.begin() + 3, 4);
    ASSERT_EQ(v.size(), 5u);
    for (size_t i = 0; i < 5; ++i) {
        EXPECT_EQ(v[i], static_cast<int>(i) + 1);
    }

    v.erase(v.begin() + 2);
    ASSERT_EQ(v.size(), 4u);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 4);
    EXPECT_EQ(v[3], 5);
}

TEST(MixedNew, RangeConstructThenInsertAndErase) {
    int arr[] = {10, 20, 30};
    kb::Vector<int> v(arr, arr + 3);

    int insert_arr[] = {15, 25};
    v.insert(v.begin() + 1, insert_arr, insert_arr + 2);
    ASSERT_EQ(v.size(), 5u);
    EXPECT_EQ(v[0], 10);
    EXPECT_EQ(v[1], 15);
    EXPECT_EQ(v[2], 25);
    EXPECT_EQ(v[3], 20);
    EXPECT_EQ(v[4], 30);

    v.erase(v.begin() + 1, v.begin() + 3);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_EQ(v[0], 10);
    EXPECT_EQ(v[1], 20);
    EXPECT_EQ(v[2], 30);
}

TEST(MixedNew, IteratorRangeAssignThenReverseIterate) {
    int arr[] = {1, 2, 3, 4, 5};
    kb::Vector<int> v;
    v.assign(arr, arr + 5);

    int expected = 5;

```

```

    for (kb::Vector<int>::reverse_iterator rit = v.rbegin(); rit != v.rend(); ++rit) {
        EXPECT_EQ(*rit, expected);
        --expected;
    }
    EXPECT_EQ(expected, 0);
}

TEST(MixedNew, EraseAllThenInsert) {
    kb::Vector<int> v(5, 10);
    v.erase(v.begin(), v.end());
    EXPECT_TRUE(v.empty());

    v.insert(v.begin(), 42);
    ASSERT_EQ(v.size(), 1u);
    EXPECT_EQ(v[0], 42);
}

TEST(MixedNew, DoubleTypeWithNewMethods) {
    double arr[] = {1.1, 2.2, 3.3};
    kb::Vector<double> v(arr, arr + 3);
    v.insert(v.begin() + 1, 1.5);
    ASSERT_EQ(v.size(), 4u);
    EXPECT_DOUBLE_EQ(v[0], 1.1);
    EXPECT_DOUBLE_EQ(v[1], 1.5);
    EXPECT_DOUBLE_EQ(v[2], 2.2);
    EXPECT_DOUBLE_EQ(v[3], 3.3);

    v.erase(v.begin() + 2);
    ASSERT_EQ(v.size(), 3u);
    EXPECT_DOUBLE_EQ(v[0], 1.1);
    EXPECT_DOUBLE_EQ(v[1], 1.5);
    EXPECT_DOUBLE_EQ(v[2], 3.3);
}

```

Диаграмма классов



Рисунок 1 — Диаграмма классов kb::Vector<T> (UML 2.0)